

APEX Overview

Audience: New researchers onboarding to the **implemented** APEX backend and API.

This document reflects the codebase under `backend/src` as of the repository state you are reading. For interface details, see `interfaces/` and `plugins/`.

What APEX Is

APEX (AI Patient Experience Simulator) is a clinical communication training system delivered as a **FastAPI** backend (`app.py`) and a separate frontend. Trainees hold text-based (and optional audio) dialogues with a simulated patient; after the session is closed, the backend produces structured **feedback** and stores **research-oriented** fields.

Problem It Addresses

- Repeated, low-risk practice for difficult conversations using **case** scripts and a **session** transcript.
 - Automated **feedback** derived from persisted turns, NLU spans, and SPIKES-related stage progression.
 - **Admin-only** anonymized exports for analysis (`ResearchService`, `/v1/research/...`).
-

Who It Is For

Role	Implementation
Trainee	Authenticated user: create/list sessions, submit turns, close session, read feedback (<code>sessions_controller</code> , <code>analytics_controller</code>).
Admin	Admin-only routes: case CRUD, all-session views, plugin registry visibility, research export (<code>admin_controller</code> , <code>research_controller</code>).

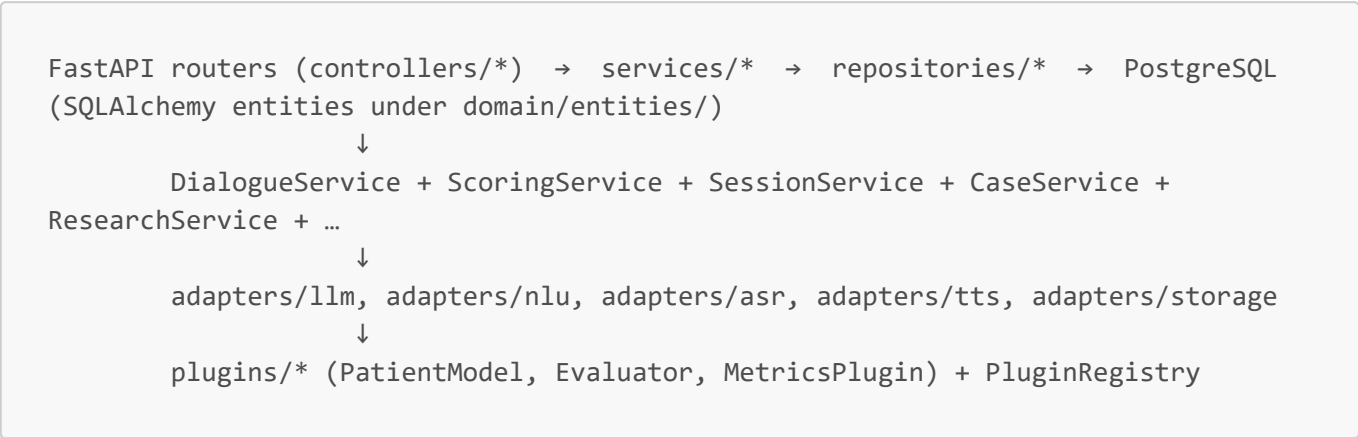
Role checks use `core/deps.py` (`get_current_user`, `require_admin`).

Implemented Core Features

- **Sessions** linked to **cases**, with reuse of an active session per (user, case) unless `force_new` (`SessionService.create_session`).
- **Turn loop** via `DialogueService.process_user_turn`: NLU on clinician text → SPIKES stage update → user turn persisted → **PatientModel** plugin generates patient text → NLU on patient text → assistant turn persisted.
- **Optional audio** turn path: Whisper ASR + tone analysis, then same dialogue pipeline (`submit_audio_turn`).
- **Optional assistant TTS** when `enable_tts` and adapters are configured.
- **Session close** → `ScoringService.generate_feedback` → persisted **Feedback**; **MetricsPlugin** results stored on `sessions.metrics_json`.

- **Research exports:** anonymized session id (`anon_` + hash), JSON and CSV endpoints under `/v1/research/`.
- **Plugin system:** `PatientModel`, `Evaluator`, `MetricsPlugin` protocols; registration at startup via `plugins/load_plugins.py`.

Layered Architecture (As in Code)



OpenAPI is mounted at `/v1/docs` (see `app.py`).

Key Domain Concepts

Concept	Implementation
Case	<code>domain/entities/case.py</code> : script, objectives, optional <code>evaluator_plugin</code> , <code>patient_model_plugin</code> , <code>metrics_plugins</code> (JSON text).
Session	<code>domain/entities/session.py</code> : state (<code>active</code> , <code>paused</code> , <code>completed</code> , <code>abandoned</code>), <code>current_spikes_stage</code> , frozen plugin fields, <code>metrics_json</code> .
Turn	User and assistant rows with <code>text</code> , <code>metrics_json</code> , <code>spans_json</code> , <code>spikes_stage</code> .
Feedback	<code>domain/entities/feedback.py</code> / <code>FeedbackResponse</code> : scores, JSON metric fields, <code>evaluator_meta</code> (includes <code>session_plugins</code> when written by <code>ScoringService._persist_feedback_from_rule_state</code>).
Dialogue	<code>DialogueService</code> + <code>NLUPipeline</code> (<code>services/nlu_pipeline.py</code>) + <code>SimpleRuleNLU</code> (<code>adapters/nlu/simple_rule_nlu.py</code>) + <code>StageTracker</code> (<code>services/stage_tracker.py</code>).
Scoring	<code>ScoringService</code> : rule-based core, <code>generate_feedback_hybrid</code> , <code>generate_feedback_hybrid_v2</code> ; invoked through Evaluator plugins or directly from <code>generate_feedback</code> .
Plugins	Protocols in <code>interfaces/</code> ; loaded by <code>load_plugins()</code> on startup; resolved from session row + <code>PluginRegistry</code> / <code>_load_class_from_path</code> .

SPIKES Stages in the Running System

During each user turn, **StageTracker** (`services/stage_tracker.py`) detects and persists stages (monotonic progression):

`setting` → `perception` → `invitation` → `knowledge` → `emotion` → `strategy`

(`DialogueService` defines a different list for legacy helpers; **live turn processing uses StageTracker.**)

One-Sentence Summary

APEX persists structured turns and SPIKES stages, generates patient replies through a **PatientModel** plugin, and on **session close** runs a frozen **Evaluator** plugin plus **MetricsPlugin** list through `ScoringService.generate_feedback`, with admin-only anonymized export of the resulting data.

Pipeline detail — `01_System_Pipeline.md`